OPTIMIZATION METHODS

Delivery Route Optimization in Colombo, Sri Lanka

A Comparative Study of Exact, Local-Search and Metaheuristic Methods

MSc-Level Programming Assignment Report

Group assignment | Two students

Student 1: MS26912448 – SENARATHNE KHIR

Student 2: MS26912370 – SERASINGHE S

Module: Optimization Methods

Abstract

A single delivery vehicle based at Colombo Fort must visit 99 localities in the Colombo, Gampaha and Kalutara districts and return to the depot. This is a Travelling Salesperson Problem (TSP). We formulate it as an integer linear program with Miller–Tucker–Zemlin (MTZ) sub-tour elimination and compare one exact method, Held–Karp dynamic programming, with five heuristics: Nearest Neighbour (NN), 2-opt, 2-opt with Or-opt, multi-start 2-opt and iterated local search (ILS). The instance uses real locality names whose coordinates were cross-checked against OpenStreetMap (OSM); distances are Haversine straight-line distances.

Held–Karp solves random instances of up to 20 nodes (5.1 s at 20 nodes, growing about five-fold for every two additional nodes). On these instances ILS reached the proven optimum in all 35 cases, NN was 11–24% above the optimum on average, and 2-opt 0.1–3.9%. On the full 100-node instance the tour length falls from 432.72 km (NN) to 334.96 km (ILS), which is 22.6% shorter, in 2.8 s. A 1-tree lower bound of 269.44 km shows that the best tour is at most 24.3% above the optimum, although the small-instance results suggest it is much closer. The main limitations are straight-line distances, approximate coordinates, a single vehicle and no capacities or time windows; the report closes with extensions to road-network distances and vehicle-routing variants.

Contents

- 1. Introduction
- 2. Problem and Dataset Description
- 3. Mathematical Formulation
- 4. Optimization Methods
- 5. Experimental Design and Evaluation
- 6. Results and Interpretation
- 7. Discussion
- 8. Limitations and Future Work
- 9. Conclusion
- 10. Individual Contributions
- References
- Appendix A: Reproducibility Checklist

1. Introduction

One of the most expensive components in urban logistics is last-mile delivery. Fuel burning, driver hours, emissions and parcels delivered per day are related to the distance a vehicle can travel. The problem becomes real when it comes to Colombo and its suburbs: there are customers spread across a city and a vast commuter belt, and a inefficient and disorganized loop can lengthen by tens of kilometres.

The beginning is the Travelling Salesperson Problem (TSP). In a given depot, and a given collection of customer locations, we want the shortest closed tour visiting each customer exactly once. A decision version of the problem is NP-complete and the number of different tours increases factorially: With 99 customers there are approximately 10^{155} of tours, so enumeration is not possible. Exact algorithms are available, and the cost increases exponentially, but there are heuristics which are cheap but not guaranteed (Applegate et al., 2006).

This project aims at modelling a realistic delivery round in Colombo, solving it exactly and with heuristic methods and comparing the quality of solutions, running times, feasibility and scalability. The contributions include: (i) a cleaned 100-node dataset whose coordinates have been validated against the OpenStreetMap data; (ii) a complete MTZ formulation and an implementation using Held–Karp methods, that were verified against brute force results; (iii) five heuristics implemented from scratch; (iv) an experimental study against proven optimal results for instances of 8–20 nodes with a comparison to the provable lower bound on the full instance; and (v) a reproducible code base and unit tests. Section 2 presents information about the data, Section 3 about the model, Section 4 about the methods, Sections 5-6 about the experiments and results, and Sections 7-9 about the discussion, limitations and conclusions.

2. Problem and Dataset Description

2.1 Operational context

A delivery company has one vehicle and one depot at Colombo Fort. In a planning period the vehicle must serve 99 customer localities spread from Kochchikade in the north to Kalutara in the south and Avissawella in the east, and return to the depot. We assume that every location is reachable, the vehicle has enough capacity, each location is visited exactly once, and no time windows or service times apply. These assumptions isolate the route-ordering problem so that the algorithms can be compared fairly. A real operator would also have several vehicles, demands, shifts and delivery windows; Section 8 discusses these.

2.2 Dataset design and source

There are 100 nodes of which node 0 is the depot (Colombo Fort) and nodes 1-99 are localities (Pettah, Nugegoda, Negombo, Gampaha, Horana and Panadura). The data for node id, name, latitude, longitude and node type are stored in the file colombo_delivery_locations.csv. The derived 100 x 100 matrix is the file

distance_matrix_km.csv. Distances are recalculated from the location file and a test ensures that they are consistent with the location matrix file.

Locality names are real localities, and the coordinates are the representative centre points from the locality, not the addresses of the customers. As real-life data is desired, each name was verified with OpenStreetMap, as detailed in Section 2.3. Distances are haversine (great circle) distances in kilometres and an Earth radius of 6371 km. This selection is an explicit approximation: the length of the routes is a lower bound of the length of the driving tours, since straight-line is shorter than driving, and distances on one-way streets are not included; the ranking of the routes might be different on a road network. The matrix is symmetric and is a triangular matrix (satisfies triangle inequality).

Table 1. Fields of the dataset and their validation.

Field	Description	Validation
node_id	Stable identifier 0–99; 0 is the depot	Unique, consecutive, no duplicates
location_name	Name of the depot or locality	No duplicate names
latitude, longitude	Representative centre coordinates (degrees)	Inside the Western Province box; no duplicate coordinates; compared with OSM
node_type	depot or delivery	Exactly one depot
cost[i,j]	Haversine distance from i to j in km	Non-negative, symmetric, zero diagonal, equal to the value recomputed from coordinates
source metadata	OSM check retrieved 14 Oct 2026 (Nominatim, ODbL)	Stored in osm_validation.csv and DATASET_NOTES.txt

2.3 Data preparation

1. The team compiled 102 locality names with approximate centre coordinates in the Colombo, Gampaha and Kalutara districts.
2. Every name was geocoded with the OpenStreetMap Nominatim service (retrieved 14 October 2026) and the distance between our point and the OSM point was recorded. 95 of 102 names returned a match; the median offset was 0.75 km, and 60, 77 and 86 of the 95 were within 1, 2 and 3 km respectively. Seven sub-locality names had no OSM match.
3. Offsets above 3 km were reviewed by hand. Two were genuine errors and were corrected to the OSM place node: Bandaragama (21.7 km off) and Ganemulla (5.2 km off). The other cases were homonyms, district centres or businesses returned by the geocoder (for example Gampaha, Kalutara and Koswatta)

and the original points were kept. “Kochchikade” was renamed “Kochchikade (Negombo)” to avoid confusion with a Colombo-15 locality of the same name.

4. Pannipitiya Junction and Moratuwa Town had exactly the same coordinates as Pannipitiya and Moratuwa. They add zero-distance nodes and no routing information, so they were removed, leaving 100 nodes. A few localities remain very close together (e.g. Narahenpita and Thimbirigasyaya, 0.11 km apart) because they are distinct places; this makes part of the instance easier than a set of well-separated customers.
5. The Haversine matrix was computed and validated: unique names and coordinates, bounding box, and consistency with the CSV file. These checks run automatically in the unit tests.
6. All algorithms use the same matrix. For the exact comparison, random subsets of 8–20 nodes (always including the depot) are drawn from the 100 nodes with fixed seeds.

3. Mathematical Formulation

3.1 Sets and parameters

Let $V = \{0, 1, \ldots, n\}$ be the set of locations, where 0 is the depot and n is the number of delivery locations ($n = 99$ in the full instance). Let c_{ij} be the travel cost from i to j in kilometres; here c_{ij} is the Haversine distance. Let x_{ij} be a binary variable equal to 1 if the vehicle drives directly from i to j and 0 otherwise. Self-arcs are excluded. For each delivery location $i = 1, \ldots, n$ let u_i be its position in the tour; these order variables are used only to forbid sub-tours.

3.2 Objective function

The objective is to minimise the total length of the closed tour:

$$\textbf{minimize } Z = \Sigma_{i \in V} \Sigma_{j \in V,\, j \neq i} c_{ij}\, x_{ij}$$

3.3 Constraints

Every location has exactly one outgoing arc and exactly one incoming arc:

$$\Sigma_{j \in V,\, j \neq i} x_{ij} = 1 \quad \text{for every } i \in V$$

$$\Sigma_{i \in V,\, i \neq j} x_{ij} = 1 \quad \text{for every } j \in V$$

The degree constraints alone allow several disconnected cycles. We eliminate sub-tours with the Miller–Tucker–Zemlin (MTZ) constraints (Miller et al., 1960):

$$u_i - u_j + n \cdot x_{ij} \leq n - 1 \quad \text{for all } i, j \in \{1, \ldots, n\},\ i \neq j$$

$$1 \leq u_i \leq n \quad \text{for } i = 1, \ldots, n; \qquad x_{ij} \in \{0, 1\} \quad \text{for all } i \neq j$$

If $x_{ij} = 1$ the first inequality forces $u_j \geq u_i + 1$, so positions increase along any path that avoids the depot. A cycle made only of delivery locations would need u_i to be larger than itself, which is impossible; therefore every feasible solution is a single Hamiltonian cycle through the depot. An equivalent but tighter formulation replaces the MTZ rows by the Dantzig–Fulkerson–Johnson constraints (Dantzig et al., 1954), $\Sigma_{i \in S} \Sigma_{j \in S, j \neq i} x_{ij} \leq |S| - 1$ for every subset S of delivery locations with $|S| \geq 2$, which are exponentially many and are normally added lazily by a solver.

The model is solved in this project by dynamic programming (Section 4.1) and not by a general ILP solver; the ILP is given because it defines the problem precisely and is the standard way to extend it with further constraints.

3.4 Assumptions and feasibility

The baseline model assumes a single vehicle, one depot, mandatory service at every delivery location, no capacity, no time windows and a static, symmetric cost matrix. A tour is feasible if it starts and ends at the depot and visits every delivery location exactly once. Every tour returned by every algorithm is checked for this in the code (the experiment runner stops with an error if a tour is infeasible).

4. Optimization Methods

4.1 Exact method: Held–Karp dynamic programming

Held–Karp (Held and Karp, 1962) stores, for every subset S of delivery locations and every j in S, the length D(S, j) of the shortest path that starts at the depot, visits exactly the locations in S and ends at j:

$$\mathbf{D(\{j\}, j) = c_{oj}}$$

$$\mathbf{D(S, j) = \min_{i \in S \setminus \{j\}} [\, D(S \setminus \{j\}, i) + c_{ij} \,]}$$

$$\mathbf{\text{optimal length} = \min_{j} [\, D(\{1, \ldots, n\}, j) + c_{j0} \,]}$$

Parent pointers recover the tour. There are $n \cdot 2^{n-1}$ states with O(n) work each, so time is $O(n^2 2^n)$ and memory $O(n 2^n)$. The implementation (src/tsp/exact.py) was verified against brute-force enumeration on random 8-node instances in the unit tests. Held–Karp was chosen because it needs no solver licence, is easy to verify and has predictable cost, which suits a scalability study. For 30 nodes (29 delivery locations) the table would already have $29 \cdot 2^{29} \approx 1.6 \times 10^{10}$ entries, and for the full 100 nodes $99 \cdot 2^{99} \approx 6 \times 10^{31}$, so the exact method is applied only to random subsets of up to 20 nodes.

For the full instance we also compute a **1-tree lower bound** (Held and Karp, 1970): the weight of a minimum spanning tree on the delivery locations plus the two cheapest depot edges. Every tour contains such a structure, so no tour can be shorter. The bound is cheap but loose, and it is used to put a provable limit on how far a heuristic tour can be from the optimum.

4.2 Constructive heuristic: Nearest Neighbour

Nearest Neighbour (NN) starts at the depot and repeatedly drives to the closest unvisited location, finally returning to the depot. It runs in $O(n^2)$, is simple and gives a baseline, but its early greedy choices leave long return trips at the end of the route and it offers no optimality guarantee (Rosenkrantz et al., 1977).

4.3 Local search: 2-opt and Or-opt

2-opt (Croes, 1958) removes two edges and reconnects the tour by reversing the segment between them whenever this shortens the tour. Because the matrix is symmetric, a move is evaluated from four distances only. The search stops when no improving move exists. 2-opt removes crossing edges but can stop at a local optimum. Or-opt (Or, 1976) moves a segment of one to three consecutive locations, optionally reversed, to a cheaper position in the tour. The two neighbourhoods repair different defects, so they are alternated until neither improves the tour ("2-opt + Or-opt").

4.4 Multi-start and iterated local search

Multi-start 2-opt runs NN from several random first customers (30 on the full instance, 10 on the small instances), improves each tour with 2-opt and keeps the best, which reduces the dependence on one greedy start. **Iterated local search (ILS)** (Lourenço et al., 2003) starts from the 2-opt + Or-opt local optimum and repeats the following: apply a random double-bridge perturbation (cut the tour into three pieces and reorder them), re-optimise with 2-opt + Or-opt, and keep the result only if the tour became shorter. This lets the search escape local optima that 2-opt and Or-opt cannot leave. ILS uses 300 iterations on the full instance and 50 on the small instances. All randomised methods use fixed seeds.

Table 2. Methods, strengths, limitations and role in the study.

Method	Strength	Limitation	Role
Held–Karp (exact)	Proves optimality	$O(n^2 2^n)$ time and memory; practical to about 20–25 nodes	Exact benchmark
Nearest Neighbour	Very fast, simple	Greedy; poor end-of-route effects	Baseline heuristic
2-opt	Removes crossing edges	Stops at a local optimum	Route improvement
2-opt + Or-opt	Also relocates badly placed stops	Still a local optimum	Stronger local search
Multi-start 2-opt	Explores several starting tours	More runtime; no guarantee	Diversification
Iterated local search	Escapes local optima; best quality	Slowest heuristic; parameters not tuned	Metaheuristic

5. Experimental Design and Evaluation

5.1 Fair comparison

All algorithms use the same locations and Haversine cost matrix. For each instance the depot, the distance units and the route-closing rule are identical. All code is plain Python 3 (standard library) and was run on an Apple-silicon MacBook Pro under macOS with Python 3.14; plots use matplotlib 3.11. Times are wall-clock seconds measured with a monotonic timer and cover only the algorithm (the matrix is built beforehand). Randomised methods use fixed seeds, so every number can be reproduced; absolute runtimes will differ on another computer, but ratios are meaningful.

5.2 Metrics

- **Objective value:** total tour length in km.
- **Runtime:** median wall-clock seconds over repeated runs.
- **Feasibility:** every returned tour is checked to start and end at the depot and to contain every node exactly once.
- **Optimality gap:** $100 \times (Z - Z^*) / Z^*$, where Z^* is the Held–Karp optimum. On the full instance no optimum is known, so we report the gap to the best tour found ("relative performance") and the gap to the 1-tree lower bound.
- **Scalability:** runtime and gap as the number of nodes grows from 8 to 20 (exact) and at 100 (heuristics).
- **Robustness:** standard deviation of the tour length over random seeds.

5.3 Experimental protocol

1. **Study A – heuristics against the proven optimum.** For each size $n \in \{8, 10, 12, 14, 16, 18, 20\}$ nodes (depot included) draw 5 random subsets of the 100 nodes with fixed seeds, solve each with Held–Karp, and run every heuristic on the same subset. Record the optimal length, runtimes and the percentage gap of each heuristic (35 instances in total).
2. **Study B – full instance.** Run all five heuristics on the complete 100-node instance. Deterministic methods are timed over 5 repeats and randomised methods run with 5 different seeds. Report best, mean and standard deviation of the length, median runtime, gap to the best tour found and gap to the 1-tree lower bound.
3. Save every result to CSV (results/scalability_runs.csv, scalability_summary.csv, full_instance_results.csv) and generate all tables and figures from those files, not by hand.

6. Results and Interpretation

6.1 Heuristics against the exact optimum

Table 3 gives the mean percentage gap to the Held–Karp optimum over 5 random subsets per size. Figure 1 shows the runtimes and Figure 2 the gaps.

Table 3. Mean gap (%) to the Held–Karp optimum; exact runtime is the median over 5 subsets (Apple-silicon MacBook Pro).

n	Exact (s)	NN	2-opt	2-opt + Or-opt	Multi-start	ILS
8	0.00021	11.04	0.06	0.00	0.06	0.00
10	0.00122	12.28	1.68	0.00	0.13	0.00
12	0.0069	11.15	0.69	0.20	0.00	0.00
14	0.03729	24.49	1.27	0.91	0.00	0.00
16	0.1943	21.13	3.43	0.00	0.19	0.00
18	1.008	14.17	2.47	0.61	0.19	0.00
20	5.058	18.34	3.87	0.51	0.31	0.00

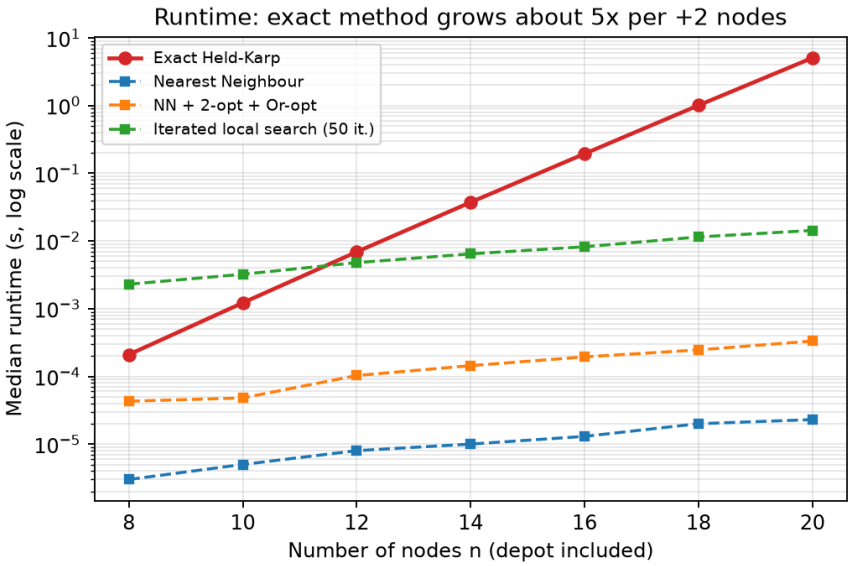


Figure 1. Median runtime against instance size (logarithmic scale). The exact method grows about five-fold for every two extra nodes.

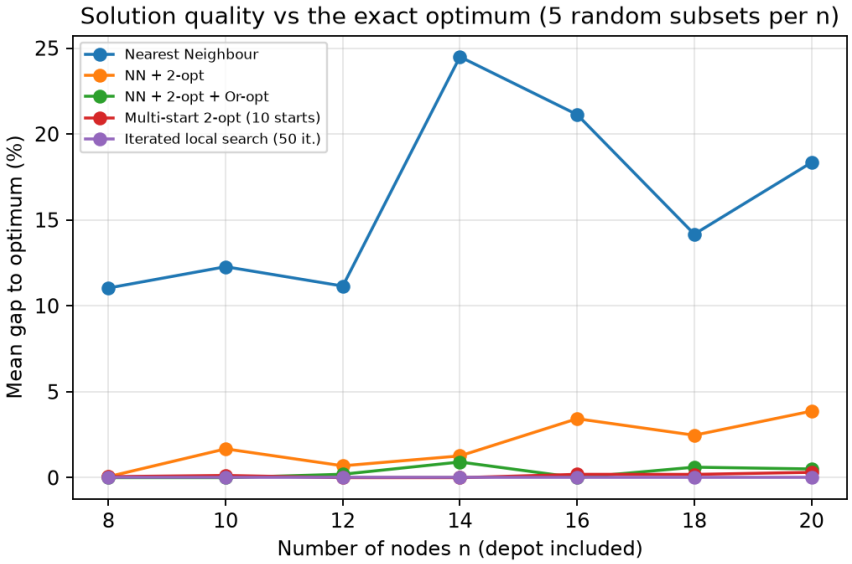


Figure 2. Mean gap to the optimum against instance size.

6.2 Full 100-node instance

Table 4. Results on the full instance (5 runs per method). Gaps refer to the best tour of each method.

Method	Best (km)	Mean (km)	Std (km)	Median time (s)	Gap to best found (%)	Gap to lower bound (%)
Nearest Neighbour	432.72	432.72	0.00	0.00019	29.19	60.60
NN + 2-opt	355.09	355.09	0.00	0.00148	6.01	31.79
NN + 2-opt + Or-opt	342.09	342.09	0.00	0.01185	2.13	26.96
Multi-start 2-opt (30 starts)	338.64	338.73	0.11	0.04689	1.10	25.68
Iterated local search (300 it.)	334.96	334.96	0.00	2.821	0.00	24.32

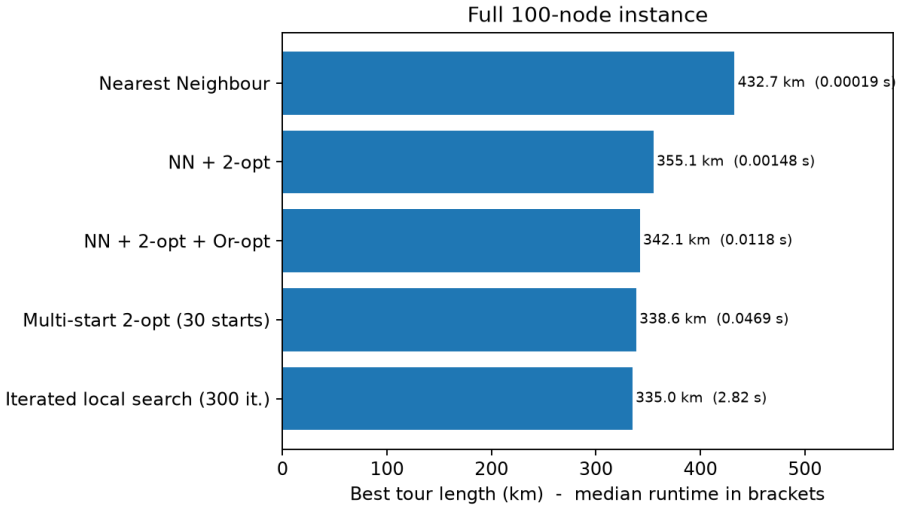


Figure 3. Best tour length and median runtime per method on the 100-node instance.

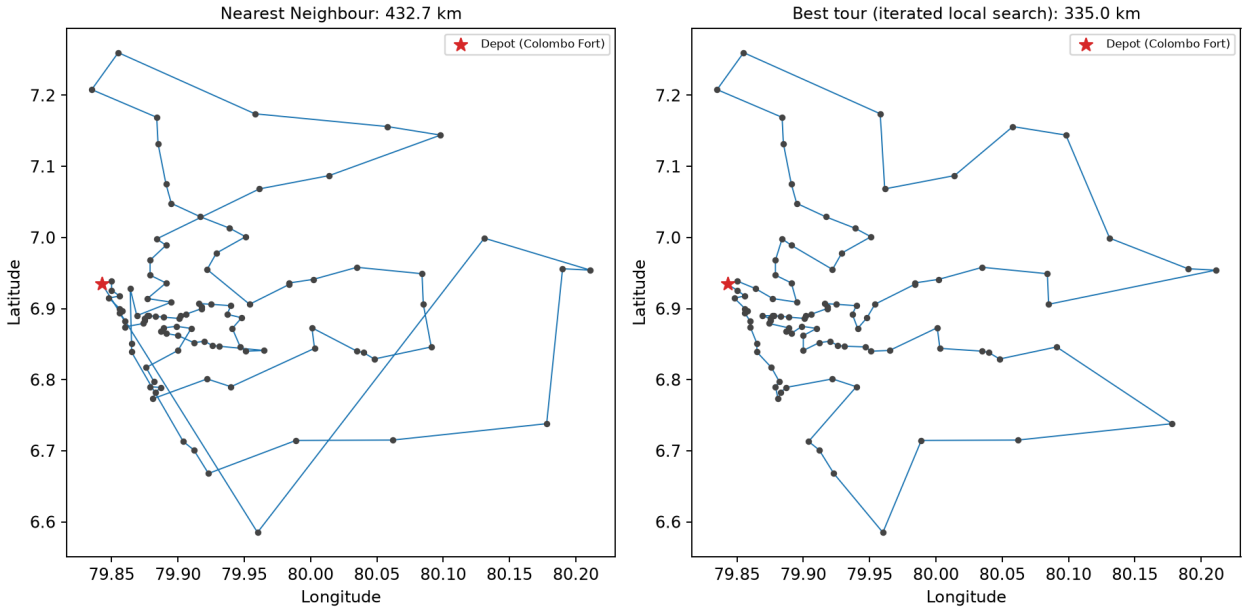


Figure 4. Nearest Neighbour tour (left) and best tour found by iterated local search (right). The depot is the star.

6.3 How to interpret the results

Improvement due to local search. Using 100 × (Z_NN − Z_improved) / Z_NN with Z_NN = 432.72 km, 2-opt shortens the NN tour by 17.9%, 2-opt + Or-opt by 20.9%, multi-start 2-opt by 21.7% and ILS by 22.6%. The NN route in Figure 4 has long crossing legs back toward the depot, which 2-opt removes.

Runtime against n. The exact method needs 0.2 ms at 8 nodes, 1.0 s at 18 nodes and 5.1 s at 20 nodes, a factor of about 5 for every two extra nodes. The heuristics grow gently: on 100 nodes NN needs 0.2 ms, 2-opt 1.5 ms, 2-opt + Or-opt 12 ms, multi-start 47 ms and ILS 2.8 s.

Stability of randomised methods. Multi-start 2-opt has a standard deviation of 0.11 km over five seeds (best 338.64 km, mean 338.73 km). ILS returned the same length, 334.96 km, for all five seeds, so it is stable on this instance.

Feasibility. 100% of the runs produced complete valid tours.

Quality against the optimum. ILS matched the proven optimum on all 35 small instances (gap 0.00% at every size). 2-opt + Or-opt had a mean gap of at most 0.9%, multi-start at most 0.3%, 2-opt between 0.06% and 3.9%, and NN between 11% and 24%. Solving small instances exactly shows that the heuristic works on those instances; it does not prove that the 100-node tour is optimal. For that instance the only provable statement is the lower bound: the best tour (334.96 km) is at most 24.3% above the 1-tree bound of 269.44 km. This bound is known to be loose, so the true gap is probably far smaller, but we do not claim a specific value.

7. Discussion

Solution quality. The ordering is as expected with the results. NN is fast but is 29% longer than the best tour known. All added ingredients contribute: 2-opt eliminates crossings (−17.9%), Or-opt improves badly placed stops (−3.0 points), multi-start and ILS require additional computation to avoid local optima. ILS provided the shortest tour, and for the cases in Study A, the optimal tour. This represents a 22.6% drop in the number of kilometers that have to be driven from NN to ILS.

Runtime and scalability. The actual method is useful as a guide but is very expensive, with 100 nodes needing about 1.6×10^{10} table entries, which is many orders of magnitude beyond the current limits. The heuristics scale polynomially. The speed of ILS is approximately 240 times slower than 2-opt + Or-opt (2.8 s vs 12 ms), but the resulting tour is 2.1% shorter. A good trade-off daily plan, or if you're looking for a real-time re-route, the fast local search might be better.

Quality of the cost matrix. Optimisation is based on realistic distances. Haversine distances have no consideration of the road network, one-way streets, congestion etc., so an optimal tour in this case may not be optimal in fact. Coordinate errors (two serious and two duplicate errors were identified) were reduced by the OSM cross-check, but coordinates are still approximate centre points. The conclusions of relative performance of the algorithms are probably going to be applicable to a road-network matrix; the absolute distances will not be applicable.

Runtime measurement. Held–Karp's time is an algorithm-only time because it is in the dynamic programme, as is the heuristics' start time. They are from one machine and one language: a compiled implementation would be faster, but the growth rates would remain the same.

Practical recommendation. Only use exact optimisation for small zones (max. 20 stops) or as reference. When it comes to larger rounds, 2-opt + Or-opt usually provides a good route in almost no time, while ILS is worthwhile for those few seconds when quality of the route is more important than speed.

8. Limitations and Future Work

The single-vehicle TSP is not representative of all delivery operations: it ignores vehicle capacity, customer demand, driver shifts, service durations and delivery time windows, and it assumes a static, symmetric cost matrix. The specific limitations of this study are:

- Distances are straight-line (Haversine) and coordinates are approximate locality centres, not geocoded customer addresses.
- The exact method was run only up to 20 nodes; the 100-node result is not proven optimal and the 1-tree bound is loose.
- Runtimes come from one machine; ILS parameters (300 iterations, double-bridge perturbation) were not tuned; Study B uses only 5 seeds.
- The 35 small test instances are random subsets of the same 100 localities, so they are not independent datasets.

Future work includes: replacing Haversine distances with road-network distances from a routing engine such as OSRM, GraphHopper or OSMnx, using a dated OSM snapshot; extending the model to the Capacitated Vehicle Routing Problem (CVRP) and to time windows (VRPTW), where the heuristics gain route-splitting and inter-route relocation moves and the ILP gains load and time variables; computing the true optimum or a much tighter bound for 100 nodes with a branch-and-cut ILP (for example through PuLP, OR-Tools or Concorde) or with a Held–Karp subgradient bound; comparing further metaheuristics such as simulated annealing (Kirkpatrick et al., 1983) and genetic algorithms; multi-objective optimisation of distance, lateness and emissions; and robust or dynamic re-optimisation under uncertain travel times and new orders.

9. Conclusion

This report modelled a single-vehicle delivery round over 99 localities around Colombo as a TSP, gave the full MTZ formulation, and compared a Held–Karp exact method with five heuristics on a 100-node dataset cross-checked against OpenStreetMap. Held–Karp solved instances of up to 20 nodes and showed exponential growth (5.1 s at 20 nodes). Iterated local search found the proven optimum in all 35 small instances and gave the shortest tour on the full instance, 334.96 km against 432.72 km for Nearest Neighbour, in 2.8 s; 2-opt with Or-opt reached 342.09 km in 12 ms. The results confirm the trade-off between optimality guarantees and scalability: exact methods certify quality but stop being practical at about 25 nodes, while local-search heuristics produce near-optimal tours at any size we tested. The main caveats are straight-line distances and approximate coordinates; the code, data and results are provided so that the study can be repeated and extended to road networks and richer routing models.

10. Individual Contributions

Table 5. Division of work between the two group members.

Area	Student 1 (MS26912448)	Student 2 (MS26912370)
Problem and data	Dataset curation, OSM cross-check, correction of errors and duplicates, data module and validation	Review of the cleaned data and its limitations
Exact method	Held–Karp, 1-tree lower bound, brute-force verification	Review and testing of the exact solver
Heuristics	–	Nearest Neighbour, 2-opt, Or-opt, multi-start, ILS and their unit tests
Experiments and figures	Design of the scalability study	Experiment runner, repeated runs, plots
Report	Sections 1–4 (problem, data, formulation, exact method)	Sections 5–9 (heuristics, experiments, results, discussion, future work)
Video	Problem, data, formulation and exact method	Heuristics, results, discussion and conclusion

The commit history of the repository records the work; commit messages are prefixed M1 (Student 1) or M2 (Student 2). Both members can explain the whole code base.

References

Applegate, D. L., Bixby, R. E., Chvátal, V., & Cook, W. J. (2006). The Traveling Salesman Problem: A Computational Study. Princeton University Press.

Croes, G. A. (1958). A method for solving traveling-salesman problems. Operations Research, 6(6), 791–812.

Dantzig, G., Fulkerson, R., & Johnson, S. (1954). Solution of a large-scale traveling-salesman problem. Operations Research, 2(4), 393–410. https://doi.org/10.1287/opre.2.4.393

Held, M., & Karp, R. M. (1962). A dynamic programming approach to sequencing problems. Journal of the Society for Industrial and Applied Mathematics, 10(1), 196–210. https://doi.org/10.1137/0110015

Held, M., & Karp, R. M. (1970). The traveling-salesman problem and minimum spanning trees. Operations Research, 18(6), 1138–1162.

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. Science, 220(4598), 671–680. https://doi.org/10.1126/science.220.4598.671

Lourenço, H. R., Martin, O. C., & Stützle, T. (2003). Iterated local search. In F. Glover & G. Kochenberger (Eds.), Handbook of Metaheuristics (pp. 320–353). Springer.

Miller, C. E., Tucker, A. W., & Zemlin, R. A. (1960). Integer programming formulation of traveling salesman problems. Journal of the ACM, 7(4), 326–329. https://doi.org/10.1145/321043.321046

Or, I. (1976). Traveling salesman-type combinatorial problems and their relation to the logistics of regional blood banking (PhD thesis). Northwestern University.

OpenStreetMap contributors. (2026). OpenStreetMap data and Nominatim geocoder, retrieved 14 October 2026. © OpenStreetMap contributors, ODbL 1.0. https://www.openstreetmap.org/copyright

Rosenkrantz, D. J., Stearns, R. E., & Lewis, P. M. (1977). An analysis of several heuristics for the traveling salesman problem. SIAM Journal on Computing, 6(3), 563–581. https://doi.org/10.1137/0206041

Appendix A: Reproducibility Checklist

Table 6. What is needed to repeat the experiments.

Item	Detail
Code and data	Repository named in submission.txt: src/tsp (data, exact, heuristics), run_experiments.py, make_plots.py, validate_data.py, tests, data/, results/. A plain-text code appendix is submitted with this report.
Dataset	data/colombo_delivery_locations.csv (100 nodes) and the derived data/distance_matrix_km.csv. OSM cross-check retrieved 14 Oct 2026 with Nominatim (data/osm_validation.csv); © OpenStreetMap contributors, ODbL.
Distance definition	Haversine great-circle distance, Earth radius 6371 km, in km. No road network and no traffic.
Software and hardware	Python 3.14 (standard library for all solvers), matplotlib 3.11 for plots; Apple-silicon MacBook Pro, macOS.
Random seeds	Subsets in Study A are drawn with seed $1000 \cdot n + s$ ($s = 0..4$); multi-start and ILS use seeds 0–4 or 1–5; all fixed in src/run_experiments.py.
Parameters	Study A: multi-start 10 starts, ILS 50 iterations. Study B: multi-start 30 starts, ILS 300 iterations; 5 repeats or seeds per method.
Commands	python -m unittest discover -s tests; python src/validate_data.py; python src/run_experiments.py; python src/make_plots.py
Verification	Held–Karp checked against brute force on 8-node instances; 1-tree bound checked against the exact optimum; every tour checked for feasibility; dataset checks in the unit tests.
Output files	results/scalability_runs.csv, scalability_summary.csv, full_instance_results.csv, best_tour.csv, best_route.txt, figures/*.png